HLD Level 2.6

Design an OTA Software Update System — Complete Interview Question & Answer

> **Interview question: Design a secure over-the-air software update system for hundreds of thousands of edge devices. The system must distribute releases safely, support staged rollout, survive interrupted downloads or installation failures, verify software integrity, and automatically roll back devices that fail after an update.**

1. Requirements

Functional: publish a software release, determine device eligibility, notify/poll devices, download artifacts, verify integrity/authenticity, install safely, reboot/restart when needed, validate health, report status, retry transient failures, pause rollout and roll back failed devices.

Non-functional: high availability, safe recovery, bounded device resource usage, secure artifacts, scalable distribution, controlled bandwidth, fleet observability and minimal device downtime.

2. Core Safety Principle

> **Never destroy the last known-good software image before the new image has been downloaded, verified, installed and proven healthy.**

For embedded/edge systems, an A/B layout is a common way to satisfy this requirement.

3. A/B Layout

```
Device Storage

Bank A -> currently active / known good
Bank B -> inactive update target

Update B while A continues to be the recovery image.
```

If A is active, install the new version into B. After validation, boot/activate B. Keep A available until B has passed post-update health checks.

4. High-Level Cloud Architecture

```
Release Engineer / CI-CD
|
v
Release Service
|
+-----+------+
v v
Metadata DB Artifact/Object Storage -> CDN
| |
+--------------+---------------+
v
Update API / Control Plane
|
v
DEVICES
```

5. Device-Side Architecture

```
Update Agent
|
+-> check release metadata
+-> validate eligibility
+-> download to temporary/inactive area
+-> verify hash/signature
+-> install inactive bank
+-> activate/reboot
+-> health validation
+-> commit OR rollback
```

```
+-> report result
```

6. Release Metadata

```
{
version,
artifact_url/id,
size,
cryptographic_hash,
signature,
compatible_device_models,
minimum_current_version,
rollout_policy,
required_disk_space
}
```

Devices should not trust an artifact merely because it came from a familiar URL. They verify signed metadata/artifacts using a trusted public key or equivalent secure trust mechanism.

7. Update State Machine

```
IDLE
-> CHECKING
-> ELIGIBLE
-> DOWNLOADING
-> VERIFYING
-> INSTALLING
-> ACTIVATING
-> VALIDATING
-> COMMITTED

Failure before commit -> RETRY or ROLLBACK/FAILED
```

Persist important state transitions so a device reboot or power loss can resume/recover deterministically rather than guessing what happened.

8. Pre-Update Validation

Before download/install, verify model/hardware compatibility, current version, free disk space, power/battery requirements, network policy, inactive-bank availability and whether another update is already in progress.

9. Download Strategy

Download to a temporary file or inactive partition rather than overwriting the active executable. Support bounded retry with exponential backoff and jitter. For large artifacts, resumable/range downloads can avoid restarting from zero after a network interruption.

10. Why CDN/Object Storage?

The control plane should not directly stream every large binary. Store immutable artifacts in object storage and distribute them through a CDN so bandwidth scales independently from release orchestration.

11. Fleet Bandwidth Example

```
500,000 devices
artifact = 200 MB

Total transfer if all update = 100,000,000 MB ~= 100 TB
```

Starting all downloads simultaneously creates enormous network/CDN/device-site pressure. Controlled rollout and randomized start times are essential.

12. Integrity and Authenticity

A cryptographic hash detects corruption; a digital signature verifies that the release was authorized by the trusted publisher. Both are important but solve different problems.

```
download -> hash verification -> signature verification -> install
```

13. Install to Inactive Bank

```
Before:
A = active v1
B = inactive old/empty

Install:
A = active v1
B = candidate v2

Do NOT overwrite A yet.
```

Installation should be transactional where possible: incomplete files, partial extraction or failed scripts must not leave the active system unusable.

14. Activation

After successful installation, mark B as the candidate boot/application bank and restart the relevant service or reboot the device depending on the platform.

15. Post-Update Health Validation

Starting the new process is not enough. Validate meaningful health signals such as process readiness, camera/driver initialization, accelerator availability, heartbeat connectivity, storage access and basic functional self-tests.

```
candidate starts
-> readiness OK?
-> heartbeat reaches backend?
-> critical hardware initialized?
-> stable for validation window?
-> COMMIT
```

16. Commit

Only after the candidate proves healthy should it become the known-good version. The previous bank can remain available for a defined rollback window or until the next safe maintenance cycle.

17. Rollback

```
v2 activation
|
health check FAILS
|
mark v2 bad
|
reactivate v1
|
restart/reboot
|
report rollback
```

Rollback must be automatic for failures that prevent reliable remote control; otherwise the fleet can become unreachable.

18. Boot Loop Protection

A candidate may repeatedly crash during boot. Maintain a boot-attempt counter or watchdog policy. After N failed boots/readiness attempts, automatically select the previous known-good bank.

19. Power Loss During Update

Design each stage to be restart-safe. If power fails during download, resume/restart the download. If it fails during inactive-bank installation, mark that bank incomplete and reinstall. Never mark the candidate committed before health validation completes.

20. Idempotency

Update operations should tolerate retries. Re-running 'download version 2' or 'install version 2 to inactive bank' should either safely continue or detect that the step already completed.

21. Staged Rollout

```
Stage 1 -> internal/test devices
Stage 2 -> 1% canary
Stage 3 -> 5%
Stage 4 -> 25%
Stage 5 -> 100%
```

At each stage, evaluate failure rate, rollback rate, crash rate, health metrics and performance before expanding.

22. Canary Selection

Select representative devices across hardware revisions, regions and network environments. Pure random selection may miss an important device class.

23. Automatic Rollout Halt

```
if rollback_rate > threshold
or crash_rate spikes
or heartbeat loss spikes
pause rollout
```

The system should be able to stop expanding a bad release without waiting for an engineer to notice manually.

24. Device Eligibility

Not every device should receive every release. Eligibility can include model, hardware revision, region, current version, feature flags, customer group and maintenance window.

25. Pull vs Push

Pull: devices periodically check for updates. Simpler through NAT/firewalls and naturally resilient, but introduces polling delay.
Push: backend notifies connected devices immediately, but requires a reliable command channel.

A hybrid is common: push a lightweight notification, then device fetches signed metadata/artifact.

26. Polling Jitter

If 500,000 devices poll exactly every hour on the hour, the backend receives a synchronized spike. Add jitter so checks are spread across the interval.

27. Retry Storm

A backend/CDN outage followed by recovery can cause every device to retry together. Use exponential backoff plus jitter and server-side admission/rate controls.

28. Status Reporting

```
device_id
current_version
target_version
state
progress
error_code
attempt_count
last_update_time
rollback_reason
```

This allows operators to answer: how many devices succeeded, failed, are downloading, are stuck, or rolled back?

29. Fleet State Storage

Maintain current update state per device in a scalable state store and append update-transition events to a durable history/analytics path.

30. Failure Categories

Download: network timeout, DNS/CDN failure, insufficient space.
Verification: checksum/signature mismatch.
Installation: filesystem permissions, extraction failure, dependency mismatch.
Activation: service/reboot failure.
Validation: process runs but hardware/network/functionality fails.
Control plane: stale metadata, unavailable API, rollout misconfiguration.

31. Error Classification

Distinguish retryable from permanent failures. Network timeout may be retryable; signature mismatch or incompatible hardware should normally stop immediately and require investigation.

32. Security

Use TLS for transport, signed release metadata/artifacts, protected signing keys, least-privilege update agents, rollback/version policies, audit logs and protection against path traversal or unsafe extraction. Avoid embedding writable private signing keys on devices.

33. Anti-Rollback

Some products must prevent installation of vulnerable older versions. Signed metadata can include a minimum allowed security version. Emergency rollback policy must be designed deliberately rather than accidentally bypassing security.

34. Update Agent Permissions

The updater often needs privileges unavailable to the main application. Give it only the filesystem/service/reboot capabilities required. OS migrations can change default filesystem or service permissions, so deployment validation must include the real production permission model.

35. Concurrency on Device

Prevent two update attempts from modifying the same bank simultaneously. Use a single update state machine/lock and keep long network downloads outside unrelated application locks.

36. Resource Limits

Bound download buffers, temporary storage, retry queues and logs. An update should not consume so much CPU, disk or network bandwidth that it breaks the primary device workload.

37. Bandwidth Throttling

On constrained devices/sites, cap download bandwidth or schedule updates during maintenance windows. Fleet control can limit concurrent downloads per region/site.

38. Observability

Track eligibility count, download starts/completions, bytes transferred, verification failures, install failures, activation failures, rollback rate, time-to-update, devices by version, stuck-state duration, retry count, CDN errors and post-update health regressions.

> **The most important rollout metric is not just 'installation succeeded'; it is whether the new version remains healthy after activation.**

39. Release-Level Dashboard

```
Release v2.3.0
Eligible: 500,000
Started: 50,000
Healthy: 48,700
Failed: 500
Rolled back: 800
Pending: 450,000
```

Break failures down by model, hardware revision, region, previous version and error category to reveal correlated problems.

40. C++ / Linux Device Agent

A C++ updater can model the workflow as an explicit state machine. Use RAII for files, sockets and temporary resources; atomic rename where applicable; fsync/durability where required; careful subprocess/systemd interaction; bounded buffers; and structured error propagation.

41. Example Device Flow

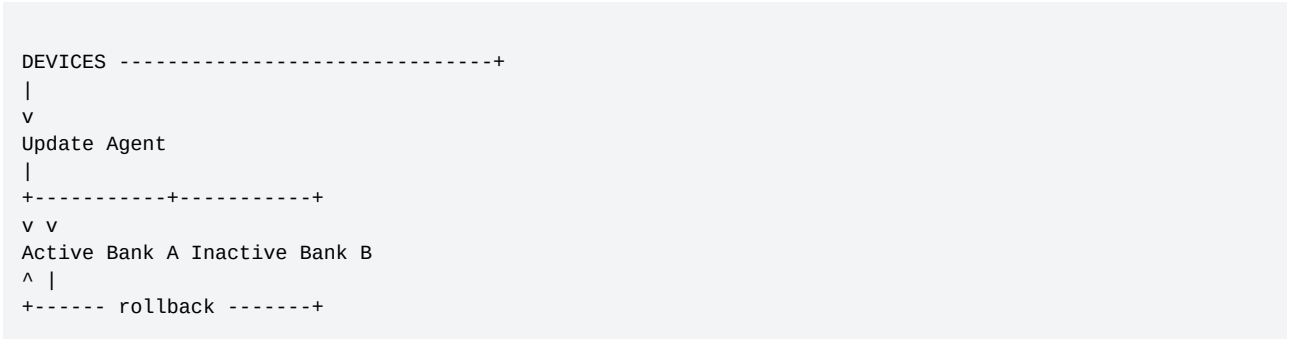
```
1. receive/check release metadata
2. verify eligibility
3. reserve/check disk space
4. download candidate to temp/inactive location
5. verify hash + signature
6. stop only services that must stop
7. install inactive bank
8. persist candidate state
9. activate/reboot
10. run readiness/self-tests
11. report health
12. commit candidate OR rollback
```

42. Graceful Interaction with Application

The updater should coordinate with the application: quiesce writes if necessary, stop services in dependency order, avoid corrupting shared files and restore services if installation aborts before activation.

43. Complete Architecture

```
RELEASE ENGINEER / CI-CD
|
v
Release Service
/ \
v v
Metadata DB Object Storage
|
v
CDN
|
+-----------------+-----------------+
| |
v v
Update Control API Status Ingestion
| ^
v |
```


DEVICES
Update Agent
Active Bank A
Inactive Bank B
rollback

44. Common Interview Follow-Ups

Why A/B? Preserve a known-good recovery image while installing the candidate.

What if download is interrupted? Resume/retry safely; active version remains untouched.

What if power fails during install? Inactive bank remains uncommitted; recover/reinstall on reboot.

When do you commit? Only after post-activation health validation.

How do you prevent fleet-wide bad release? Canary/staged rollout with automatic halt thresholds.

How do you prevent 500k simultaneous downloads? Rollout waves, randomized scheduling, CDN and regional/site concurrency limits.

How verify security? TLS plus cryptographic hashes and signed release metadata/artifacts.

What if new version boot-loops? Boot-attempt/watchdog threshold automatically returns to known-good bank.

45. Interview-Ready Spoken Answer

"I'd design the update system around a control plane and a safe device-side state machine. The backend stores signed release metadata and immutable artifacts in object storage/CDN, while the control plane manages eligibility and staged rollout. On the device, I would use an A/B strategy so the currently running known-good image is never overwritten. The update agent checks compatibility and disk space, downloads the candidate into the inactive bank, verifies its cryptographic hash and signature, installs it, persists the candidate state and activates it. After restart, the device must pass meaningful readiness and functional health checks before the candidate is committed. If activation or validation fails, the device automatically rolls back to the previous bank. At fleet scale, I would use canary rollout, randomized start times, exponential backoff with jitter and CDN distribution so hundreds of thousands of devices do not download or retry simultaneously. I would track failure and rollback rates by hardware model, region and previous version and automatically pause rollout if health regresses. On the C++/Linux device side, I'd implement an explicit restart-safe state machine, use RAII for resources, bounded buffers, least-privilege updater permissions and careful systemd/service coordination."

46. Five Decisions to Remember

1. Never destroy known-good software -> A/B BANKS.
2. Downloaded does not mean trusted -> HASH + SIGNATURE VERIFICATION.
3. Started does not mean healthy -> POST-ACTIVATION VALIDATION BEFORE COMMIT.
4. Never update the whole fleet first -> CANARY + STAGED ROLLOUT + AUTO-HALT.
5. Network/power/process failures will happen -> PERSISTED, IDEMPOTENT, RESTART-SAFE STATE MACHINE + ROLLBACK.

47. Final Revision Flow

```
Requirements
-> A/B safety model
-> release metadata/artifact storage/CDN
-> eligibility
-> download
-> hash/signature verification
-> inactive-bank install
-> activation
-> health validation
-> commit/rollback
-> staged rollout
-> retry/jitter/bandwidth control
-> security/permissions
-> observability
-> C++/Linux state machine
```